



List Problem Bank — ArrayList & LinkedList (Java)

Legend

★ = Core Placement Problem (Mandatory)

Covers internal implementation details alongside application-level problems.

These problems should be completed by every student before moving to Set.

Section 1: List Basics and Creation

★ Create and Display an ArrayList

★ Create and Display a LinkedList

★ Add Elements using add()

★ Access Elements using get()

★ Find Size of a List using size()

Create an Immutable List using List.of()

★ Convert Array to ArrayList using Arrays.asList()

Convert ArrayList to Array using toArray()

Section 2: ArrayList vs LinkedList Internals (Core Concept)

★ Internal Structure of ArrayList (Dynamic Array, Backing Array)

★ Internal Structure of LinkedList (Doubly Linked Node Structure)

★ Demonstrate ArrayList Capacity Growth / Resizing Behavior

★ Time Complexity Comparison: get(index) — ArrayList vs LinkedList

★ Time Complexity Comparison: add()/remove() at Beginning — ArrayList vs LinkedList

★ When to Use ArrayList vs LinkedList (Practical Demonstration)

Memory Overhead Comparison (Node Pointers vs Contiguous Array)

★ Fail-Fast Iterator Behavior (ConcurrentModificationException Demo)

Section 3: Adding and Removing Elements

★ Add Element at a Specific Index

★ Remove Element by Value

★ Remove Element by Index

Remove All Elements using clear()

★ Remove Elements Matching a Condition using removeIf()

Add All Elements from Another List using addAll()

Section 4: Searching and Iterating

★ Search for an Element using contains()



Find Index of an Element using indexOf()

- ★ Iterate using a Traditional for Loop
- ★ Iterate using Enhanced For-Loop
- ★ Iterate using Iterator
- ★ Iterate using ListIterator (Bidirectional Traversal)

Iterate using forEach() with Lambda

Section 5: Sorting and Ordering

- ★ Sort a List using Collections.sort()
- ★ Sort a List in Descending Order
- ★ Sort a List using a Custom Comparator
- ★ Sort a List of Custom Objects using Comparable

Sort a List using Comparator.comparing() (Method Reference)

Section 6: List Conversion and Utility

- ★ Convert List to Set to Remove Duplicates

Convert List to Array

- ★ Reverse a List using Collections.reverse()

Shuffle a List using Collections.shuffle()

Create a Sublist using subList()

Section 7: List-Based Logical Problems

- ★ Find Maximum and Minimum in a List
- ★ Remove Duplicates while Preserving Order
- ★ Find Common Elements between Two Lists
- ★ Check if a List is a Palindrome
- ★ Rotate a List Left by K Positions

Merge Two Sorted Lists

Find the Second Largest Element in a List

Section 8: Real World List Applications

- ★ Student Marks List Analyzer
- ★ Shopping Cart Total Calculator using ArrayList
- ★ Employee List Processor (List of Custom Objects)
- ★ To-Do List Manager using LinkedList

Undo/Redo Feature Simulation using LinkedList